

Laboratory field guide

Jason Karpeles | Constructed exercises and conditional calculations

Choose a working route

Start with the mathematical question. The notebooks teach a hand calculation, a changed assumption, and a transfer task. Chapter skills apply the same local Python method to supplied JSON. The master skill chooses compatible chapter methods; a phrase suggestion still needs an assumption check.

Read first, then execute

The local HTML guide at `guide/index.html` is immediately readable without Python. PDF pages are reading editions and do not execute their printed results. For a first calculation, run `notebooks/00-start-here.ipynb`. For the integrated workflow, run `notebooks/28-document-release-capstone.ipynb`.

Setup and recovery

This laboratory accompanies *The Mathematics of AI Agents* by Jason Karpeles. It contains a notebook and an assistant skill for every chapter, a master skill, an orientation notebook, a complete document-release example, and a standalone workbook with separate solutions.

The examples use declared local models and fictitious data. They teach calculations and diagnosis; they do not measure a deployed language model or document service. Your own compatible inputs can replace the examples without changing the mathematical method.

Read without installing anything

1. Extract the complete ZIP. Keep its folders together.
2. Open `guide/index.html` in a browser.
3. Select **Start with one successful calculation**. The expected-utility example should show release preferred at probability 0.8 and abstention preferred at probability 0.6.
4. Choose a chapter, inspect its executed calculation and plots, then attempt its questions before opening the separate solutions.

These are reading pages. Their code and outputs are visible, but changing a page does not execute Python. Equations render from local assets; no account or remote mathematics service is needed. Chapter 18 also has a local browser mock-up, `Companion/release-console.html`, linked from the guide index.

Run the laboratory on Mac

1. Double-click **Open Laboratory.command** in the extracted folder. A small text menu should appear. If macOS reports that the file is from an unidentified developer or cannot be opened, Control-click the file, choose **Open**, and confirm once; or open Terminal in the extracted folder and run `python3 tools/launch_lab.py`, which shows the same menu. The file is not signed, so this prompt is expected.
2. Choose **Run a chapter example** and enter a number from 1 to 27. A calculated report should appear. This requires Python 3.10 or later but no notebook packages.
3. To edit and run notebook cells, choose **Install notebook tools** once. This creates a dedicated `.venv` inside the extracted folder. It does not modify system Python. The tested notebook stack requires Python 3.11 or later; first setup may need a network connection to obtain dependencies. The package list is `requirements-notebooks.txt` (numpy, matplotlib, nbformat, nbclient, ipykernel and JupyterLab); the launcher installs it for you.
4. Choose **Open Jupyter notebooks**. In the browser, open `notebooks/00-start-here.ipynb`, then select **Run**  **Run All Cells**. The utility checks should finish without errors. JupyterLab is installed as version 4.x without an exact pin, and the delivery did not test its browser transport, so report any launch problem.

The launcher uses the dedicated environment when one exists. Keep the folder in a stable location after installing skills. You can keep your own input files and reports in a separate `reader-output` folder.

Windows and Linux

On Windows, double-click **Open Laboratory.bat** with a supported Python installed. If Python is not found, the file opens the reading guide and tells you to install Python. The same menu offers examples, setup and notebooks. You can always read `guide/index.html` without Python.

On Linux, or when using a terminal on any platform, run these commands from the extracted folder:

```
python3 tools/launch_lab.py
```

For notebook setup:

```
python3 tools/setup_lab.py
```

On Windows, use `py -3` in place of `python3` if that is how Python was installed. Platform-specific launch and installation acceptance is reported in the verification record; successful Python calculations do not establish a tested Windows or Linux graphical launch.

Use chapter skills with an assistant

1. In the launcher, choose **Install the 28 skills for Codex**.
2. Read the displayed installation directory and confirm the installation. Existing skills with matching names are preserved.
3. Restart the assistant so it can refresh skill discovery.

4. Ask: “Use `$math-ai-agents` to compare verification with retry using this trace.” Supply the trace when the method needs it. For a teaching example, ask explicitly for a constructed example.

The master selects methods by the mathematical question. Chapter skills use the same computations as the notebooks. Each skill has an input contract and worked use cases. A report should contain a result, assumptions, limitations, and an execution receipt when a computation actually ran.

The launcher targets Codex’s user skill directory. Other assistants can read the same `SKILL.md` instructions, but their discovery mechanisms may differ. No MCP server, hosted model or API key is required by these calculations. Installing a skill does not supply authorization to publish, change a live system, or contact a reviewer.

For uninstalling, choose **Remove unmodified installed skills**. Skills you changed are retained, and notebook files and reader data are not removed.

Bring your own inputs

Every chapter has a JSON example in `data/examples/`. Its notebook explains each field. Copy that example to a new file and replace the values. Do not overwrite the original fixture if you want to preserve the worked result.

The notebook’s **Apply the method to your inputs** cell reads a local file and computes its report. Change `reader_file` to your copy, then run the cell. A valid JSON input is not evidence that its probabilities are calibrated, its runs representative, or its authority current. Those remain part of your contract.

A chapter helper can also run without Jupyter:

```
python3 skills/maa-06-expected-utility/scripts/run.py --input my-choice.json --output reader-\
output/choice-report.json
```

The master helper lists available methods, suggests routes, executes selected methods, and runs the integrated capstone:

```
python3 skills/math-ai-agents/scripts/run.py list
python3 skills/math-ai-agents/scripts/run.py route "Analyze the lost acknowledgement and retry"
python3 skills/math-ai-agents/scripts/run.py capstone --output reader-output/capstone.json
```

Route suggestions are phrase matches. The assistant still needs to check the intended result and the method’s assumptions. Composed workflows report methods separately; they do not silently combine incompatible scores into a guarantee.

Recover from setup problems

What happened	Action and expected result
Python cannot be found	Read the guide immediately; install a supported Python before running calculations. Reopen the launcher afterward.
Notebook setup stopped during installation	Check the connection and rerun Install notebook tools . Existing notebooks and inputs remain intact.
A notebook cannot find the bundle	Open it from the complete extracted folder rather than copying one notebook alone. The bundle manifest, content and source folders must remain together.
The assistant says its runtime is missing	Restore the extracted folder to its installed location, or reinstall skills from its new location after reviewing the existing skill copies. The installer preserves collisions rather than overwriting them.
An input is rejected	Compare the field names, types, probability sums and units with the chapter's input contract. Quoted Boolean strings are not Boolean values.
A result is unavailable or null	Check whether the data identify that quantity. Zero successes, absent pairs or missing costs do not become zero estimates.
A changed parameter appears to do nothing	Restart the notebook kernel and run cells in order. Check which experiment input was changed and whether the method actually uses that field.

Verification and limits

The delivery's notebook execution receipt identifies the actual interpreter, fresh-kernel method, code cells and hashes. Independent mathematical checks and skill-use reviews are reported separately. This environment's socket restrictions mean normal browser/server notebook launch is a different acceptance scope from the executed notebook cells. Human reader acceptance and any untested host or operating system remain explicitly unperformed.

For details about the notebook format and execution, see the official [Jupyter notebook format](#), [notebook execution documentation](#), and [JupyterLab installation guide](#).

Chapter methods and useful outputs

Each method below uses declared constructed inputs or compatible local reader data. Its result remains conditional on its probability model, data boundary, units, and permission contract.

01. When a Score Becomes a Skill

Compare continuous evidence and thresholded scores without attributing a mechanism.

Notebook: `notebooks/01-score-threshold.ipynb`. Skill: `maa-01-score-threshold`.

Ask about: audit sudden benchmark score; threshold measurement cliff; continuous versus exact-match jump.

02. How to Measure Emergence

Compute the signed four-cell contrast, positive amount, and valid share.

Notebook: `notebooks/02-four-cell-interaction.ipynb`. Skill: `maa-02-four-cell-interaction`.

Ask about: matched four-cell component ablation; positive joint interaction share; ablation budget confounding.

03. How to Predict Emergence

Fit a declared curve on development data and score a separate test.

Notebook: `notebooks/03-prospective-forecast.ipynb`. Skill: `maa-03-prospective-forecast`.

Ask about: freeze prospective capability forecast; development test leakage audit; score unseen curve prediction.

04. The Path from Prediction to Action

Return reachable states, a permitted path, and sinks under frozen permissions.

Notebook: `notebooks/04-permission-reachability.ipynb`. Skill: `maa-04-permission-reachability`.

Ask about: allowed action graph reachability; permission bridge removal; reachable workflow sinks.

05. What the Model Becomes Inside an Agent

Compose a chooser and tool kernel, then separate marginal from trajectory reliability.

Notebook: `notebooks/05-composite-kernel.ipynb`. Skill: `maa-05-composite-kernel`.

Ask about: compose frozen chooser tool kernel; conditional versus marginal trajectory reliability; shared failure long-run construction.

06. The Price of a Choice

Return an expected-utility choice and a break-even value under declared stakes.

Notebook: `notebooks/06-expected-utility.ipynb`. Skill: `maa-06-expected-utility`.

Ask about: compare declared action utility; break-even stakes decision; price abstention alternative.

07. The Equation That Looks Ahead

Compute a finite-horizon optimal policy and its value by backward induction.

Notebook: `notebooks/07-finite-horizon-planning.ipynb`. Skill: `maa-07-finite-horizon-planning`.

Ask about: backward induction finite policy; future reward stopping decision; remaining horizon Bellman value.

08. Acting in the Dark

Update a belief and calculate gross and net one-step information value.

Notebook: `notebooks/08-belief-information.ipynb`. Skill: `maa-08-belief-information`.

Ask about: Bayesian hidden-state belief update; one-step value of observation; uninformative signal aliasing.

09. Searching the Future

Execute A* and audit supplied heuristic admissibility and cost.

Notebook: `notebooks/09-astar-audit.ipynb`. Skill: `maa-09-astar-audit`.

Ask about: A-star heuristic admissibility audit; search expansion and heuristic cost; misleading future-cost estimate.

10. Plans Within Plans: Hierarchical Actions and Reusable Skills

Compute option returns with duration discounting and explicit completion.

Notebook: `notebooks/10-option-duration.ipynb`. Skill: `maa-10-option-duration`.

Ask about: variable-duration option return; skill interruption termination contract; option deadline continuation discount.

11. The Mathematics of Curiosity

Compare greedy, UCB, and Thompson choices in a declared stationary bandit.

Notebook: `notebooks/11-bounded-exploration.ipynb`. Skill: `maa-11-bounded-exploration`.

Ask about: bounded UCB exploration budget; Thompson posterior bandit sampling; pseudo-regret versus information value.

12. Credit for Consequences

Compare Monte Carlo, TD(0), and accumulating TD(λ) updates on one trajectory.

Notebook: `notebooks/12-trajectory-credit.ipynb`. Skill: `maa-12-trajectory-credit`.

Ask about: Monte Carlo TD credit comparison; eligibility trace consequence update; terminal versus truncated bootstrap.

13. Learning to Choose: Training an Agent from Trajectories

Train a seeded softmax policy with REINFORCE and evaluate a separate success predicate.

Notebook: `notebooks/13-finite-policy-learning.ipynb`. Skill: `maa-13-finite-policy-learning`.

Ask about: REINFORCE verifier mismatch; seeded softmax policy learning; terminal reward shaping audit.

14. Building a World Inside

Calculate uniform transition error, actual fixed-policy finite values, and conditional bounds.

Notebook: notebooks/14-transition-model-bound.ipynb. Skill: maa-14-transition-model-bound.

Ask about: uniform total variation model audit; fixed-policy planning error bound; horizon model-error sensitivity.

15. What an Agent Should Remember: Retrieval, Compression, and Experience

Choose fresh valid records and inspect action preservation under compression.

Notebook: notebooks/15-memory-budget.ipynb. Skill: maa-15-memory-budget.

Ask about: retrieval knapsack token budget; stale version-bound memory authority; decision-preserving compression.

16. Thought as Search

Separate candidate coverage from actual selector success and choose a feasible allocation.

Notebook: notebooks/16-sample-allocation.ipynb. Skill: maa-16-sample-allocation.

Ask about: candidate selector compute allocation; shared-error sampling saturation; sample budget deadline frontier.

17. State and Consequence

Separate effects from acknowledgements and price a next recovery step.

Notebook: notebooks/17-effect-and-retry.ipynb. Skill: maa-17-effect-and-retry.

Ask about: lost acknowledgement tool effect; price verification versus retry; idempotency key duplicate trace.

18. When an Action Has Coordinates

Compare coordinate, semantic, and version-bound action contracts.

Notebook: notebooks/18-interface-freshness.ipynb. Skill: maa-18-interface-freshness.

Ask about: coordinate stale layout action; semantic versus version-bound interface; current permission observation age.

19. When Another Mind Becomes Part of the World

Compute diagonal, cross-play, and deployment-mixture performance.

Notebook: notebooks/19-cross-play-transfer.ipynb. Skill: maa-19-cross-play-transfer.

Ask about: self-play cross-play partner transfer; counterparty distribution policy audit; supervisor mixture coordination.

20. Messages, Beliefs, and Consensus

Enumerate bounded message worlds and distinguish agreement from local knowledge.

Notebook: notebooks/20-bounded-message-knowledge.ipynb. Skill: maa-20-bounded-message-knowledge.

Ask about: bounded message receipt knowledge; dropped acknowledgement consensus trace; local-view handoff agreement.

21. Markets, Teams, and Institutions

Compare Braess equilibrium, social optimum, tolls, and overhead.

Notebook: `notebooks/21-congestion-incentives.ipynb`. Skill: `maa-21-congestion-incentives`.

Ask about: Braess workflow congestion; private equilibrium social routing cost; toll institution incentive design.

22. Safe Enough to Act

Compare penalty choice with hard authorization and expected-risk constraints.

Notebook: `notebooks/22-risk-authority-contract.ipynb`. Skill: `maa-22-risk-authority-contract`.

Ask about: expected risk versus permission boundary; penalty constrained action audit; hard authorization safe choice.

23. When the Environment Gives Instructions: The Mathematics of Agent Security

Replay a fictitious attack trace against capability, provenance, and review checks.

Notebook: `notebooks/23-local-security-monitor.ipynb`. Skill: `maa-23-local-security-monitor`.

Ask about: untrusted data control promotion; capability provenance monitor trace; stale review adversarial publish.

24. How Much Capability Have We Extracted?

Report observed denominators, matched differences, uncertainty assumptions, and target mixtures.

Notebook: `notebooks/24-matched-capability.ipynb`. Skill: `maa-24-matched-capability`.

Ask about: paired procedure capability evaluation; reweight target task mixture; observed exposure subset denominator.

25. Improving an Agent Without Trusting the Improvement

Select on development and check only the frozen winner against a declared guard contract.

Notebook: `notebooks/25-development-guard-gate.ipynb`. Skill: `maa-25-development-guard-gate`.

Ask about: development winner guard release; adaptive guard contamination; freeze improvement acceptance procedure.

26. How Much More Could the System Become?

Separate bank oracle coverage, actual choice, and permission-filtered success.

Notebook: `notebooks/26-conditional-bank-ceiling.ipynb`. Skill: `maa-26-conditional-bank-ceiling`.

Ask about: finite candidate bank oracle ceiling; actual selector capability extraction; permission-filtered deployment coverage.

27. The Mathematics of Delegation

Compute a review queue diagnostic and identify required handoff packets and release blockers.

Notebook: `notebooks/27-delegation-queue.ipynb`. Skill: `maa-27-delegation-queue`.

Ask about: human delegation review capacity; authority handoff queue deadline; M-M-1 review saturation.

Reliability needs the right evidence

Use Chapter 5 for a declared step or composite-kernel construction, Chapter 17 for uncertain tool consequences, Chapter 16 for candidate and selector allocation, and Chapter 24 for observed run-level outcomes. A marginal step average does not identify a trajectory law. A selected candidate does not prove its effect was authorized or confirmed.

Read the result boundary

Every report should name the executed method, input contract, evidence kind, assumptions, limitations, and unavailable quantities. A constructed example teaches what a model implies; it does not measure deployed-agent performance. Human usability testing, operating-system launch acceptance, and assistant discovery are different checks from mathematical execution.